



Rusty on Google Cloud

A managed, serverless setup for up to 500 KHEL students on the Bihar Board curriculum. Everything that stores data sits in Mumbai (`asia-south1`). There are no servers to patch and no load balancer to pay for.

- Plan only • nothing provisioned
- Prices checked 29 Sep 2026
- Staging + production

~\$110 per month, typical use (≈ ₹9,700)	500 students, ~150 at peak	Mumbai region <code>asia-south1</code> for all data	5 managed services, 0 VMs
--	--	---	---

21
DAYS

The configured AI model retires on 20 October 2026. The code calls `gemini-2.5-flash-lite` through `vertexai.generative_models` , which Google removed from the Vertex AI SDK in June 2026. Both have to change before any production use (checklist item B2).

#	DECISION	CHOICE	WHY
D1	Region	<code>asia-south1</code> (Mumbai)	Closest region to Bihar with every service needed. Keeps student data in India.
D2	Backend	Cloud Run service	Billed per request, scales on demand. The Dockerfile already targets it.
D3	Frontend	Firebase Hosting	Free at this size. Global CDN and free SSL.
D4	Database	Cloud SQL PostgreSQL 16 + <code>pgvector</code> , <code>db-g1-small</code>	Managed backups and patching. 60k vectors fit easily.
D5	Users & flags	Firestore + Firebase Auth	What the code already uses. Close to free.
D6	PDF ingestion	Cloud Run job	Background tasks stall on Cloud Run once the response is sent.

#	DECISION	CHOICE	WHY
D7	LLM	gemini-3.1-flash-lite , global	Cheapest current Gemini. Needs a KHEL decision on where processing happens.
D8	Embeddings	gemini-embedding-001 at 1024	Handles Hindi and fits the existing column. The configured model stops at 768.
D9	Networking	No VPC, no load balancer	Saves \$20+ a month. The database is reachable only through the IAM connector.
D10	Environments	Two projects: staging and prod	Staging scales to zero for about \$10-15 a month.
D11	Deploys	GitHub Actions + Workload Identity	No key files in GitHub, as CLAUDE.md requires.

NEEDS A DECISION FROM KHEL LEADERSHIP

Where may the AI process questions?

The cheapest Gemini models do not run in India. The one that does costs about six times more. Switching later takes two settings, VERTEX_AI_LOCATION and GEMINI_MODEL .

Option A · Flash-Lite, global

Recommended for cost

~\$60 / month for the AI

- gemini-3.1-flash-lite , GA, supported until at least May 2027.
- \$0.25 in / \$1.50 out per million tokens.
- Google gives no control over which country processes a request.
- Leaves India: the question, up to 10 earlier turns, textbook passages. No student ID, name or IP address.

Option B · Flash, Mumbai

In-country processing

~\$380 / month for the AI

- gemini-3.5-flash in asia-south1 , supported until at least May 2027.
- \$1.65 in / \$9.90 out per million tokens.
- Every request stays in India.
- A stronger model. Heavy use could reach about \$1,000 a month.

Ask your data-protection adviser. Check whether sending pseudonymous study questions from minors outside India fits your DPDP Act 2023 duties and parental consent terms. If the answer is unclear, start the pilot on Option B and move to A later. This page is not legal advice.

ESTIMATE · USD PER MONTH · PRODUCTION

What it will cost

Fixed costs are about \$50 a month. The AI is the only line that grows with use.

Adjust the numbers to match your centres.

Usage preset

Light **Typical** Heavy

AI model

A · Flash-Lite, global

B · Flash, Mumbai

Students active per day **150**



Questions per student per day **6**



Practice tests per student per day **0.5**



School days per month **26**

☐

Let the API sleep at night (saves ~\$6, first morning request is slow)

\$108 ≈ ₹9,500 / month



Vertex AI Gemini (answers + tests)	\$58
Cloud SQL database	\$32
Cloud Run API (one warm instance)	\$15
Storage, registry, secrets, Firestore, embeddings	\$4

Each question costs about \$0.0021 and each test about \$0.0046 on option A.

Printed with the Typical preset on option A. Open the online page to change the numbers. Assumes ~3,500 input tokens per request, ~800 output tokens per answer and ~2,500 per test. Hosting and Auth stay within free tiers. Add ~\$10–15 for staging. Rate ≈ ₹88 per USD.

Ways to keep the bill down

1. Daily cap per student, for example 40 questions and 5 tests.
2. The response cache already skips the AI for repeat questions.
3. Lowest thinking level and tight output limits per mode.
4. A lower Vertex AI quota as a hard ceiling. Budgets only alert.
5. Budget alerts at 50%, 90% and 100% of \$150.

Staging costs about \$10–15

Staging uses the smallest database (db-f1-micro), lets the API sleep, and sends only test traffic to the AI. Stop the staging database when nobody is testing.

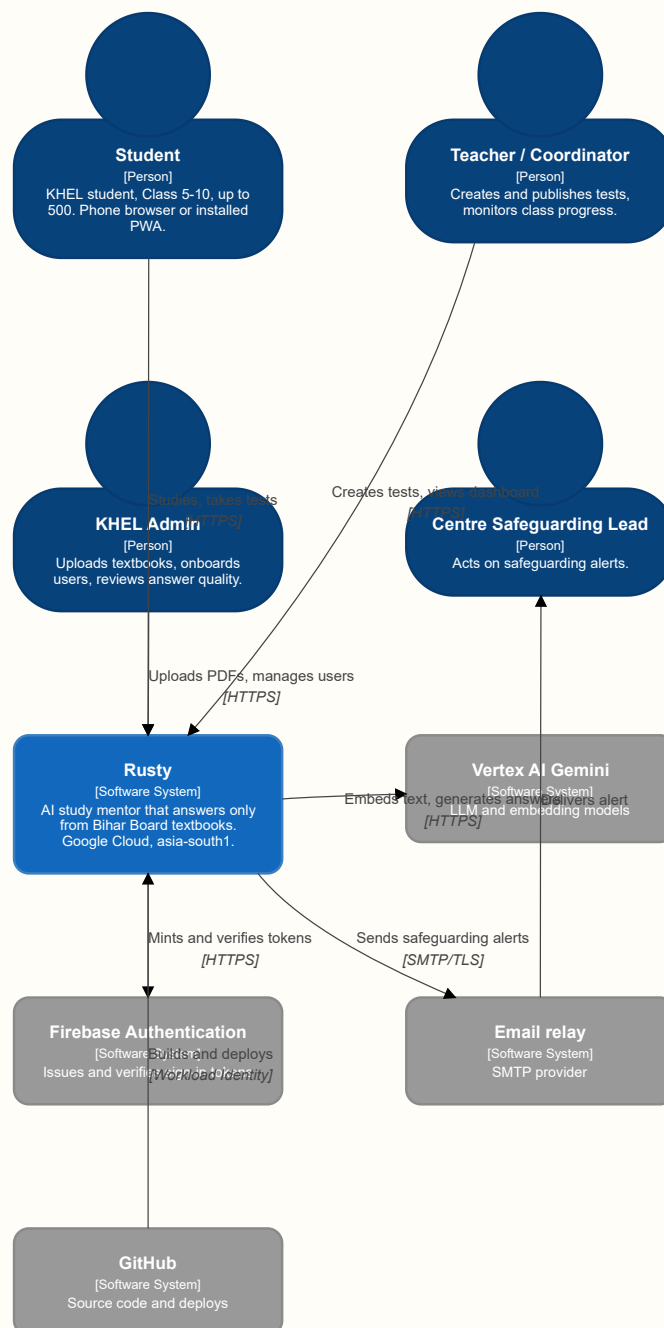
```
gcloud sql instances patch rusty-d  
b --activation-policy=NEVER
```

C4 MODEL

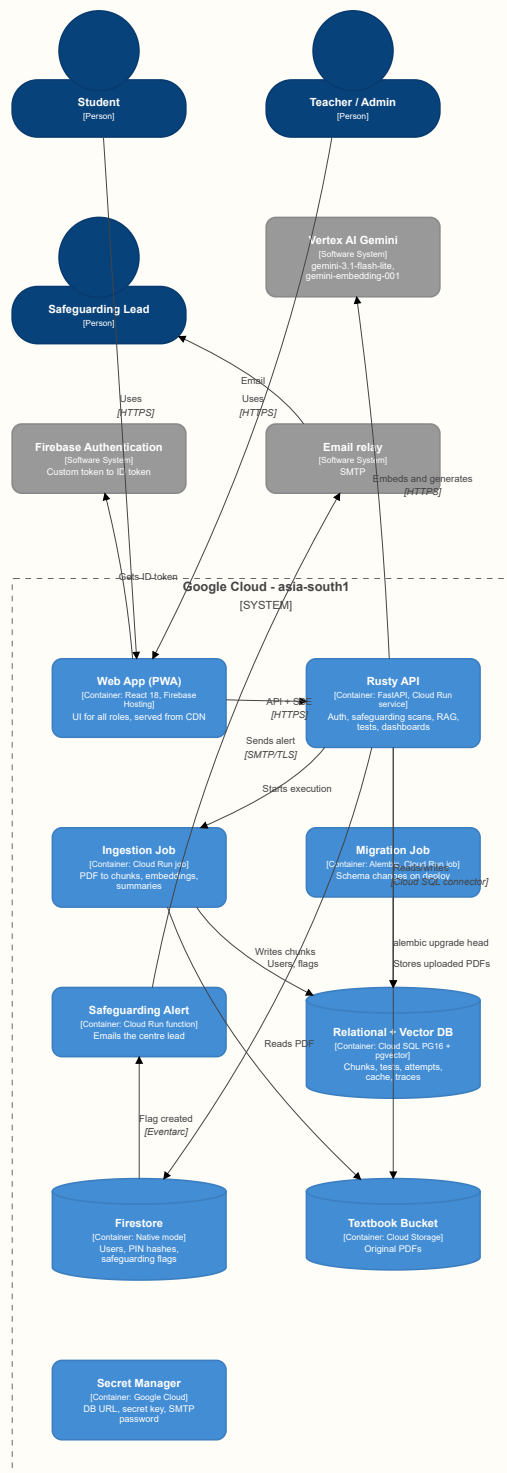
Architecture

Read top to bottom: who uses Rusty, the parts it is built from, where each part runs, and what happens when a student asks a question.

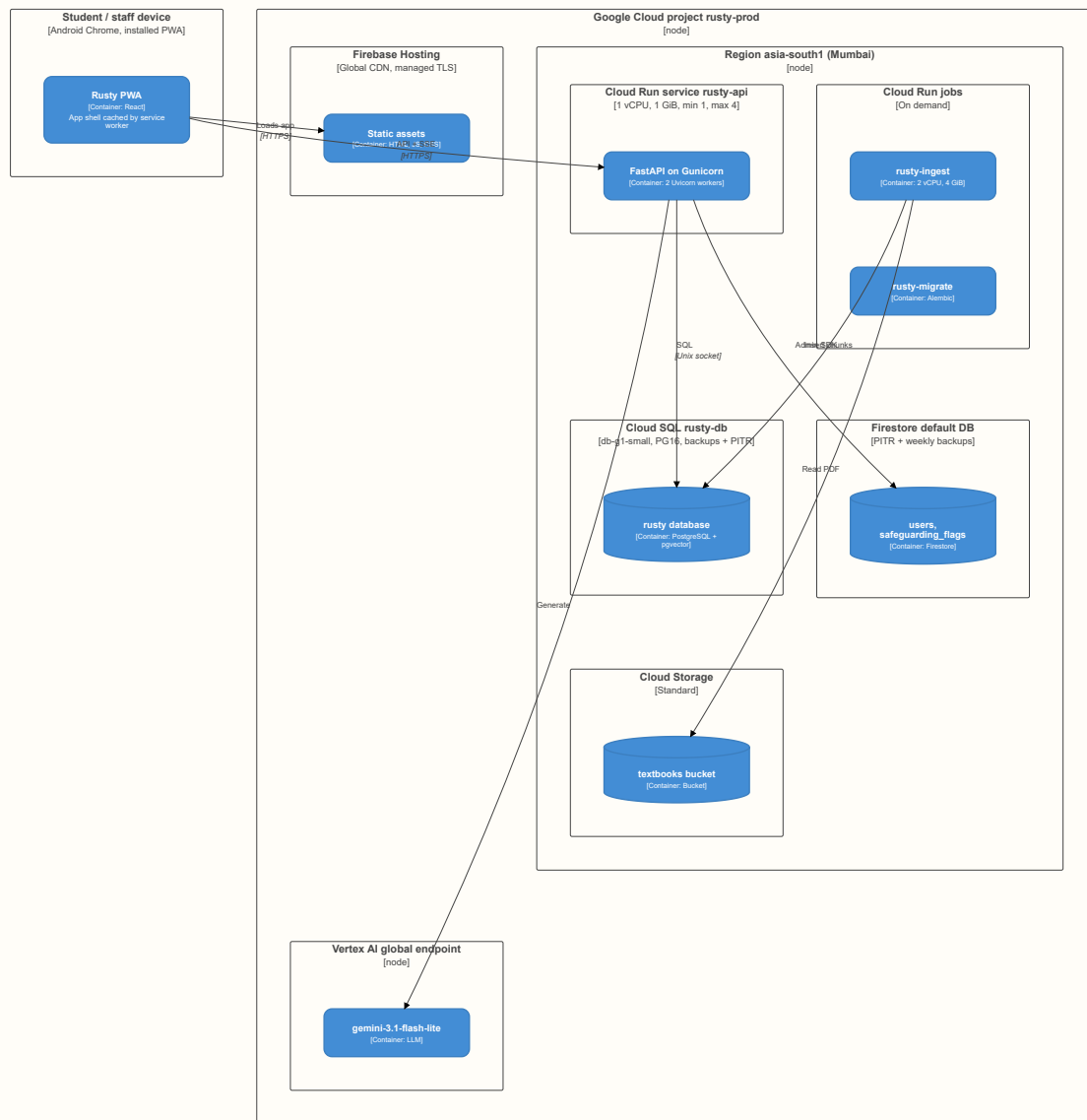
System Context: Rusty AI Study Mentor



Containers: Rusty on Google Cloud (rusty-prod)

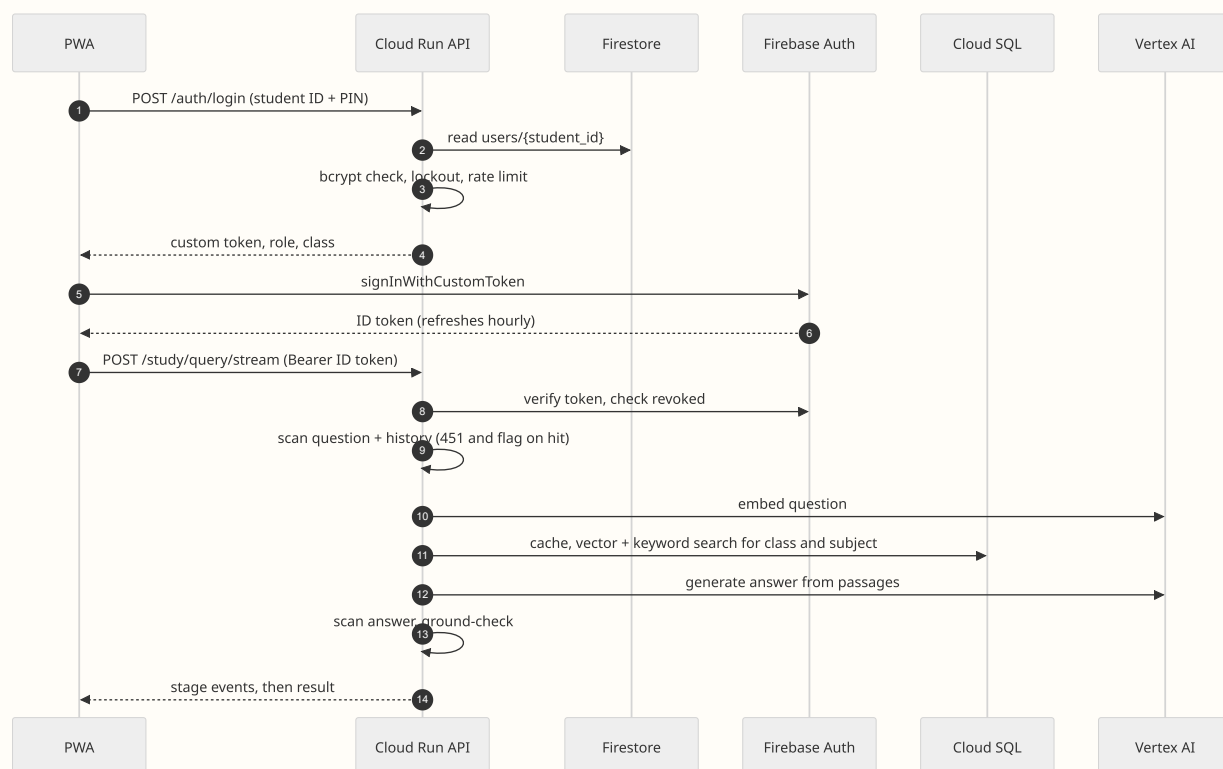


Deployment: Production (rusty-prod)



Flow · Login and one study question

Where each safeguarding check happens



CONFIGURATION

Components

Each part, how it is sized, and its typical monthly cost.

Rusty API

\$15

Cloud Run service · `rusty-api`

Size 1 vCPU, 1 GiB, 2 Gunicorn workers

Scaling min 1, max 4, 40 requests each

Timeout 120 s, startup CPU boost on

Access Public URL; the app checks Firebase tokens

One warm instance removes 5–15 second cold starts. Four instances cap both spend and database connections.

Database

\$32

Cloud SQL · PostgreSQL 16 + pgvector

Tier `db-g1-small`, 1.7 GB RAM, zonal

Storage 10 GB SSD, grows automatically

Backups Daily at 02:00 IST, 7 kept, PITR 7 days

Access No authorised networks; IAM connector only

About 50 connections allowed. Set the pool to 3 + 2 per worker so four instances use 40 at most.

Gemini

\$58

Vertex AI · Option A shown

LLM `gemini-3.1-flash-lite`,
global

Embeddings `gemini-embedding-001`, 1024
dims

Settings Low thinking level, JSON
schema, temp 0.1–0.2

SDK `google-genai`

Use the same embedding model for
textbooks and questions. Every PDF is re-
ingested on the first deploy.

Users & safeguarding

\$1

Firestore + Firebase Authentication

Location `asia-south1`, delete protection

Backups PITR + weekly, kept 14 weeks

Rules Deny all browser access

Sign-in All providers off; custom tokens
only

The API's service account needs Token
Creator on itself, or login fails while signing
tokens.

Web app

\$0

Firebase Hosting

Build `VITE_API_URL` + Firebase web
config

Headers CSP, nosniff, strict referrer

Caching Assets immutable; `index.html` and
`sw.js` no-cache

Domain `*.web.app` or your own, free SSL

The browser calls the Cloud Run URL directly.
Hosting rewrites time out at 60 seconds and
don't suit streaming.

Jobs & supporting services

\$3

Cloud Run jobs, Storage, Artifact Registry, Secret
Manager

Ingest `rusty-ingest`, 2 vCPU, 4 GiB, 60
min

Migrate `rusty-migrate`, runs before each
release

PDFs Kept in a private bucket for re-
ingestion

Secrets `DATABASE_URL`, `SECRET_KEY`, SMTP

The jobs reuse the API image, so each
release is one build.

ENVIRONMENT VARIABLE

PRODUCTION VALUE

NOTE

`APP_ENV`

`production`

Any other value accepts the dev
tokens, including `dev-admin-
token`.

`LLM_PROVIDER`

`vertex`

`VERTEX_AI_LOCATION`

`global`

`asia-south1` for Option B.

`GEMINI_MODEL`

`gemini-3.1-flash-lite`

`gemini-3.5-flash` for Option
B.

ENVIRONMENT VARIABLE	PRODUCTION VALUE	NOTE
<code>VERTEX_EMBED_MODEL</code> / <code>EMBED_DIM</code>	<code>gemini-embedding-001</code> / <code>1024</code>	Must match <code>vector(1024)</code> .
<code>FIREBASE_*_EMULATOR_HOST</code>	unset	Setting either points production at an emulator.
<code>ALLOWED_ORIGINS</code>	Hosting domains only	No wildcard.
<code>DB_POOL_SIZE</code> / <code>DB_MAX_OVERFLOW</code>	<code>3</code> / <code>2</code>	New setting. Keeps the database under 50 connections.
<code>TEXTBOOK_BUCKET</code> / <code>INGEST_JOB_NAME</code>	<code><project>-textbooks</code> / <code>rusty-ingest</code>	New settings for the ingestion job.

CODE WORK · TICKS ARE SAVED IN THIS BROWSER ONLY

Before the first deploy

IDs such as C1 or P4 refer to findings in `PROJECT_ANALYSIS.md` .

0 of 17 done

A · Safeguarding

● Before any real student

- ☐ A1
- Scan AI output on the streaming result, cache hits and chapter summaries
Analysis C1, C2
- ☐ A2
- Write a Firestore flag on every blocked message, and build the alert function that emails the flag ID, centre and time
Analysis C3
- ☐ A3
- Match whole words, remove over-broad phrases, grow test cases to 100+ including false positives such as "grape"
Analysis C4

B · Deploy blockers

● App will not run on GCP without these

- ☐ B1
- Replace the two macOS symlinks, build the image from the repo root, add `pymupdf` and `google-genai`
Analysis P4, P6

☐ B2 Move the LLM client to the `google-genai` SDK, map the `assistant` role to `model`, default to `gemini-3.1-flash-lite`
Analysis P2 · model retires 20 Oct 2026

☐ B3 Use one embedding model for textbooks and questions (`gemini-embedding-001` at 1024 with task types). Summaries go through the shared LLM client.
Analysis P3

☐ B4 Frontend signs in with the custom token and sends the Firebase ID token
Analysis P1

☐ B5 Regenerate migrations from the models; skip `create_all` in production
Analysis P5

☐ B6 Upload PDFs to Cloud Storage and start the `rusty-ingest` job instead of a background task
New

☐ B7 Make database pool size configurable
New

☐ B8 Rate-limit per signed-in student. A whole centre shares one IP, and Cloud Run hides it behind Google's proxy.
New

☐ B9 Refuse to start on Cloud Run unless `APP_ENV=production`
New

C · Usable pilot

● Before students are invited

☐ C1 A real way to onboard students: admin endpoints, or at least the bootstrap script in Operations
Analysis F1

☐ C2 Save published-test attempts on the server, stop sending answer keys, fill the teacher dashboard
Analysis F2, F3

☐ C3 One answer per question per attempt; students locked to their own class
Analysis F4, F5

☐ C4 Stop the PWA caching study, test and admin responses; bundle KaTeX CSS locally
Analysis medium



C5

Daily question and test cap per student

Cost control

RUN IN CLOUD SHELL · STAGING FIRST, THEN PRODUCTION

Setup steps

Cloud Shell is the terminal button in the Google Cloud console. It already has `gcloud`, `docker`, `node` and `firebase` signed in as you, so nothing needs installing on Windows.

0

Accounts and billing

Console, one time

1. Choose the Google accounts for the tech leads and turn on 2-step verification.
2. Console → Billing → create a billing account and add a payment method.
3. If KHEL uses Google Workspace, create the projects under the organisation and grant access through groups.

1

Set variables

Edit the first three lines

```
export PROJECT_ID="rusty-prod-khel"           # globally unique
export REGION="asia-south1"
export BILLING_ACCOUNT="XXXXXX-XXXXXX-XXXXXX" # gcloud billing accounts list
export ALERT_EMAIL="safeguarding-lead@example.org"
export API_SA="rusty-api@${PROJECT_ID}.iam.gserviceaccount.com"
export JOBS_SA="rusty-jobs@${PROJECT_ID}.iam.gserviceaccount.com"
export IMAGE="${REGION}-docker.pkg.dev/${PROJECT_ID}/rusty/api"
export SQL_CONN="${PROJECT_ID}:${REGION}:rusty-db"
export BUCKET="${PROJECT_ID}-textbooks"
```

2

Project, APIs and budget

Creates the project and a \$150 budget alert

```
gcloud projects create "$PROJECT_ID" --name="Rusty Prod"
gcloud billing projects link "$PROJECT_ID" --billing-account="$BILLING_ACCOUNT"
gcloud config set project "$PROJECT_ID"

gcloud services enable \
  run.googleapis.com sqladmin.googleapis.com artifactregistry.googleapis.com \
  secretmanager.googleapis.com aiplatform.googleapis.com firestore.googleapis.com \
  storage.googleapis.com iam.googleapis.com iamcredentials.googleapis.com sts.googleapis.com \
  cloudbuild.googleapis.com firebase.googleapis.com firebasehosting.googleapis.com \
  identitytoolkit.googleapis.com eventarc.googleapis.com cloudfunctions.googleapis.com \
  monitoring.googleapis.com logging.googleapis.com billingbudgets.googleapis.com

gcloud billing budgets create --billing-account="$BILLING_ACCOUNT" \
  --display-name="rusty-prod monthly" --budget-amount=150USD \
  --filter-projects="projects/${PROJECT_ID}" \
  --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 --threshold-rule=percent=1.0
```

3

Image registry

Private Docker repository in Mumbai

```
gcloud artifacts repositories create rusty --repository-format=docker \
  --location="$REGION" --description="Rusty container images"
```

In the console, add a cleanup policy that keeps the 10 most recent versions.

4

Service accounts and permissions

Least privilege for the API and the jobs

```
gcloud iam service-accounts create rusty-api --display-name="Rusty API runtime"
gcloud iam service-accounts create rusty-jobs --display-name="Rusty jobs runtime"

for ROLE in roles/cloudsql.client roles/datastore.user roles/aiplatform.user \
           roles/firebaseauth.viewer roles/logging.logWriter roles/monitoring.metricWriter; do
    gcloud projects add-iam-policy-binding "$PROJECT_ID" \
        --member="serviceAccount:${API_SA}" --role="$ROLE" --condition=None
done
for ROLE in roles/cloudsql.client roles/aiplatform.user roles/logging.logWriter; do
    gcloud projects add-iam-policy-binding "$PROJECT_ID" \
        --member="serviceAccount:${JOBS_SA}" --role="$ROLE" --condition=None
done

# Lets firebase-admin sign custom tokens on Cloud Run without a key file
gcloud iam service-accounts add-iam-policy-binding "$API_SA" \
    --member="serviceAccount:${API_SA}" --role="roles/iam.serviceAccountTokenCreator"
```

5

Database

Cloud SQL with backups and point-in-time recovery

```
gcloud sql instances create rusty-db \
    --database-version=POSTGRES_16 --edition=ENTERPRISE --tier=db-g1-small \
    --region="$REGION" --availability-type=zonal \
    --storage-type=SSD --storage-size=10 --storage-auto-increase \
    --backup-start-time=20:30 --retained-backups-count=7 \
    --enable-point-in-time-recovery \
    --maintenance-window-day=SUN --maintenance-window-hour=21 \
    --deletion-protection
# Times are UTC: backups at 02:00 IST, maintenance Monday 02:30 IST.

gcloud sql databases create rusty --instance=rusty-db
DB_PASS="$(openssl rand -base64 30 | tr -d '/+= ' | cut -c1-32)"
gcloud sql users create rusty_app --instance=rusty-db --password="$DB_PASS"
```

6

Secrets

Database URL and app key in Secret Manager

```
printf 'postgresql+asyncpg://rusty_app:%s@/rusty?host=/cloudsql/%s' "$DB_PASS"
"$SQL_CONN" \
| gcloud secrets create DATABASE_URL --data-file=-
openssl rand -hex 32 | tr -d '\n' | gcloud secrets create SECRET_KEY --data-file
=-
unset DB_PASS

for S in DATABASE_URL SECRET_KEY; do
  gcloud secrets add-iam-policy-binding "$S" \
    --member="serviceAccount:${API_SA}" --role="roles/secretmanager.secretAccess
or"
done
gcloud secrets add-iam-policy-binding DATABASE_URL \
  --member="serviceAccount:${JOBS_SA}" --role="roles/secretmanager.secretAccesso
r"
```

7

Firestore

Delete protection, recovery and weekly backups

```
gcloud firestore databases create --location="$REGION" --type=firestore-native -
-delete-protection
gcloud firestore databases update --database='(default)' --enable-pitr
gcloud firestore backups schedules create --database='(default)' \
  --recurrence=weekly --day-of-week=SUN --retention=14w
```

8

Firebase

Console steps plus deny-all rules

1. Firebase console → *Add project* → pick the existing GCP project.
2. Authentication → *Get started*. Leave every sign-in provider off. Keep only the Hosting domains under *Authorised domains*.
3. Project settings → *Your apps* → add a web app named `rusty-web`. Copy its config into GitHub as the `VITE_FIREBASE_*` values.
4. Cloud console → APIs & Services → Credentials: limit that browser key to your Hosting domains and to the Identity Toolkit and Token Service APIs.
5. Create `firestore.rules` that denies all browser access, add it to `firebase.json`, then deploy it.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} { allow read, write: if false; }
  }
}
```

```
firebase use "$PROJECT_ID" && firebase deploy --only firestore:rules
```

9

Textbook bucket

Private storage for uploaded PDFs

```
gcloud storage buckets create "gs://${BUCKET}" --location="$REGION" \
  --uniform-bucket-level-access --public-access-prevention
gcloud storage buckets add-iam-policy-binding "gs://${BUCKET}" \
  --member="serviceAccount:${API_SA}" --role="roles/storage.objectCreator"
gcloud storage buckets add-iam-policy-binding "gs://${BUCKET}" \
  --member="serviceAccount:${JOBS_SA}" --role="roles/storage.objectViewer"
```

10

Build and push the image

Needs checklist item B1

```
git clone <your-repo-url> rusty && cd rusty
gcloud auth configure-docker "${REGION}-docker.pkg.dev" --quiet
TAG="$(git rev-parse --short HEAD)"
docker build -f backend/Dockerfile -t "${IMAGE}:${TAG}" .
docker push "${IMAGE}:${TAG}"
```

11

Migration job

Create once, run on every release

```
COMMON_ENV="^|^APP_ENV=production|LLM_PROVIDER=vertex|VERTEX_AI_PROJECT=${PROJECT_ID}|VERTEX_AI_LOCATION=global|GEMINI_MODEL=gemini-3.1-flash-lite|VERTEX_EMBED_MODEL=gemini-embedding-001|EMBED_DIM=1024|FIREBASE_PROJECT_ID=${PROJECT_ID}|SAFEGUARDING_ALERT_EMAIL=${ALERT_EMAIL}|PHRASES_FILE_PATH=./safeguarding/phrases_v1.json|TEXTBOOK_BUCKET=${BUCKET}|DB_POOL_SIZE=3|DB_MAX_OVERFLOW=2"
# "^|^" tells gcloud to split on | so values may contain commas.
```

```
gcloud run jobs create rusty-migrate --image="${IMAGE}:${TAG}" --region="$REGION" \
  --service-account="$JOBS_SA" --set-cloudsql-instances="$SQL_CONN" \
  --set-secrets="DATABASE_URL=DATABASE_URL:latest" \
  --set-env-vars="$COMMON_ENV|SECRET_KEY=unused-by-migrations" \
  --command="alembic" --args="upgrade,head" --task-timeout=10m --max-retries=0

gcloud run jobs execute rusty-migrate --region="$REGION" --wait
```

12

Ingestion job

Needs checklist item B6

```
gcloud run jobs create rusty-ingest --image="${IMAGE}:${TAG}" --region="$REGION" \
  --service-account="$JOBS_SA" --set-cloudsql-instances="$SQL_CONN" \
  --set-secrets="DATABASE_URL=DATABASE_URL:latest" \
  --set-env-vars="$COMMON_ENV|SECRET_KEY=unused-by-jobs" \
  --command="python" --args="-m,rag.ingestion.job" \
  --cpu=2 --memory=4Gi --task-timeout=60m --max-retries=1

gcloud run jobs add-iam-policy-binding rusty-ingest --region="$REGION" \
  --member="serviceAccount:${API_SA}" --role="roles/run.jobsExecutorWithOverrides"
```

13

Deploy the API

Then check /health and /ready

```
gcloud run deploy rusty-api --image="${IMAGE}:${TAG}" --region="$REGION" \
  --service-account="$API_SA" --add-cloudsql-instances="$SQL_CONN" \
  --set-secrets="DATABASE_URL=DATABASE_URL:latest,SECRET_KEY=SECRET_KEY:latest" \
  --set-env-vars="$COMMON_ENV|INGEST_JOB_NAME=rusty-ingest|ALLOWED_ORIGINS=https://${PROJECT_ID}.web.app,https://${PROJECT_ID}.firebaseapp.com" \
  --cpu=1 --memory=1Gi --concurrency=40 --min-instances=1 --max-instances=4 \
  --timeout=120 --cpu-boost --allow-unauthenticated

API_URL="$(gcloud run services describe rusty-api --region="$REGION" --format='value(status.url)')"
curl -s "$API_URL/health"
curl -s "$API_URL/ready"
```

14

Drop request logs that contain IP addresses

Required by CLAUDE.md

Cloud Run's own request logs record each student's IP. The app writes its own request log with a hashed IP, so the platform logs can go.

```
gcloud logging sinks update _Default \
  --add-exclusion="name=exclude-run-request-logs,filter=logName=\"projects/${PROJECT_ID}/logs/run.googleapis.com%2Frequests\""
```

15

Deploy the web app

Put the API host into the CSP first

```
cd frontend && npm ci
VITE_API_URL="$API_URL" VITE_FIREBASE_API_KEY=... VITE_FIREBASE_AUTH_DOMAIN="${PROJECT_ID}.firebaseapp.com" \
VITE_FIREBASE_PROJECT_ID="$PROJECT_ID" VITE_FIREBASE_APP_ID=... npm run build
cd .. && firebase deploy --only hosting
```

16

Monitoring and alerts

Console

1. Monitoring → Uptime checks: HTTPS to the Cloud Run host, path `/health`, every 5 minutes, email alert.
2. Logging → Log-based metrics: count `jsonPayload.event="safeguarding_tier1_triggered"` and alert the safeguarding lead and tech lead on any hit.
3. Do the same for `circuit_breaker_tripped`.
4. Alerting: Cloud Run 5xx above 5% for 10 minutes; Cloud SQL CPU above 80% for 15 minutes, disk above 80%, more than 40 connections.

17

Go-live checks

All must pass on staging first

- Upload one textbook as admin and watch `rusty-ingest` finish with status *ready*.
- Run `python -m eval.run_eval --base-url "$API_URL"`; it should pass at least 10 of 11.
- A test phrase returns 451, writes a Firestore flag, sends the email and fires the log alert.
- `dev-admin-token` gets 401 in production.
- Cloud Run request logs no longer appear in Logs Explorer.
- Budget alert emails arrive.
- One database restore into staging has been practised.

GITHUB ACTIONS

Releases

A push to `staging` deploys to staging. A push to `main` waits for a reviewer to approve the production environment. GitHub signs in to Google through Workload Identity Federation, so no key file exists anywhere.

STAGE	WHAT RUNS
Checks	Existing <code>ci.yml</code> : tests, truffleHog, pip-audit, npm audit, type check. Required on both branches.
Build	One image from the repo root, tagged with the commit SHA, pushed to Artifact Registry.
Migrate	Point <code>rusty-migrate</code> at the new image and run it. Migrations must work with the previous release still serving.

STAGE	WHAT RUNS
Deploy API	Update <code>rusty-ingest</code> and deploy <code>rusty-api</code> . Settings and secrets carry over.
Deploy web	Build with the <code>VITE_*</code> variables and run <code>firebase deploy --only hosting</code> .

```
# One-time setup: deployer identity and the GitHub trust link
gcloud iam service-accounts create github-deployer --display-name="GitHub Actions dep
loyer"
DEPLOYER="github-deployer@${PROJECT_ID}.iam.gserviceaccount.com"
for ROLE in roles/run.developer roles/artifactregistry.writer roles/firebasehosting.a
dmin; do
  gcloud projects add-iam-policy-binding "$PROJECT_ID" --member="serviceAccount:${DEP
LOYER}" --role="$ROLE" --condition=None
done
for SA in "$API_SA" "$JOBS_SA"; do
  gcloud iam service-accounts add-iam-policy-binding "$SA" --member="serviceAccoun
t:${DEPLOYER}" --role="roles/iam.serviceAccountUser"
done
gcloud iam workload-identity-pools create github --location=global --display-name="Gi
tHub"
gcloud iam workload-identity-pools providers create-oidc github-provider \
  --location=global --workload-identity-pool=github \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=assertion.re
pository,attribute.ref=assertion.ref" \
  --attribute-condition="assertion.repository=='<github-owner>/Rusty-AI-Study-Mento
r'"
PROJECT_NUMBER="$(gcloud projects describe "$PROJECT_ID" --format='value(projectNumbe
r)')")
gcloud iam service-accounts add-iam-policy-binding "$DEPLOYER" \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/gl
obal/workloadIdentityPools/github/attribute.repository/<github-owner>/Rusty-AI-Study-
Mentor"
```

The full `deploy.yml` workflow is in `GCP_DEPLOYMENT_PLAN.md`, section 9.3.

DAY TO DAY

Operations

Roll back a bad release

```
gcloud run revisions list --service=rusty-api --region=asia-south1
gcloud run services update-traffic
rusty-api --region=asia-south1 \
  --to-revisions=<previous-revision>=100
firebase hosting:rollback
```

Database changes are not undone automatically. Ship a fix forward, or restore from a point in time if data was damaged.

Onboard students

1. Prepare a CSV of student ID, class, centre and role. No names.
2. A tech lead runs the bootstrap script in Cloud Shell with their own sign-in.
3. It creates a random 6-digit PIN per student, stores only the bcrypt hash in Firestore, and marks the account to change its PIN.
4. Print the PIN slips, hand them out, then delete the file.

When a safeguarding alert arrives

1. The centre lead follows the KHEL safeguarding policy. The email carries the flag ID.
2. The designated safeguarding administrator reads the details in Firestore `safeguarding_flags`.
3. Flags are never deleted. Mark them resolved and record who resolved them.

Restore the database

```
gcloud sql instances clone rusty-db
rusty-db-restore \
  --point-in-time="2026-10-01T10:15:00Z"
```

Check the data, point the `DATABASE_URL` secret at the clone, add it to the service's Cloud SQL instances, and redeploy.

WHEN YOU SEE	DO THIS
Database CPU above 70% for days, or you need an SLA	Move to <code>db-custom-1-3840</code> (dedicated vCPU, about \$50–60 a month)
Frequent 429s or all 4 instances busy	Raise max instances and the database connection budget together
More than 1,000 students or several centres	High-availability database, a task queue for ingestion, shared rate limits
Need a custom API domain or a web firewall	Add a load balancer with Cloud Armor (about \$20–25 a month more)
Setup by hand gets tiring	Write these steps as Terraform in <code>infra/</code>

How each rule is enforced

RULE	ENFORCED BY	STATUS
Closed access, ID + PIN only	No sign-in providers; backend mints tokens; Firestore user record checked on every call	Needs B4
Every input and output scanned	Code, plus a log alert on every trigger	Needs A1-A3
No conversation history on the server	History stays in the browser; consider Vertex zero data retention	Met
Query text visible to admins only	Stored in rag_traces , 180-day expiry	Teachers can read it today
No IP or unmasked student ID in logs	App hashes both; platform request logs excluded (step 14)	Needs step 14
No names, ages or contacts stored	Onboarding CSV has IDs and class only	Met
No hardcoded secrets or key files	Secret Manager, Workload Identity, truffleHog in CI	Met
Container never runs as root	USER rusty in the Dockerfile	Keep in B1
No CORS wildcard in production	ALLOWED_ORIGINS lists Hosting domains; config rejects *	Met
Safeguarding flags never deleted	No delete path, delete protection, recovery, 14-week backups, browser access denied	Needs A2

FOR PEOPLE NEW TO GCP

Glossary

Project	Holds resources, billing and permissions. One per environment.	Region	A data-centre location. asia-south1 is Mumbai.
---------	--	--------	--

Cloud Run service	Runs your container over HTTPS and adds or removes copies with demand.	Cloud Run job	Runs a container to completion, then stops.
Revision	A fixed version of a service. Rolling back sends traffic to an older one.	Cloud SQL	PostgreSQL where Google handles patches and backups.
Firestore	Document database holding users and safeguarding flags.	Firebase Hosting	Serves the React build from a global CDN.
Artifact Registry	Private storage for Docker images.	Secret Manager	Encrypted passwords and keys, given to the app at start-up.
Service account	An identity for code, granted permissions through IAM roles.	Workload Identity	Lets GitHub act as a service account without a key file.
Vertex AI	Google's AI platform that serves Gemini.	Eventarc	Delivers events, such as a new Firestore document, to a function.
PITR	Point-in-time recovery: restore to any second within the window.	Cloud Shell	A free browser terminal with Google's tools already installed.

CHECKED 29 SEP 2026

Sources

- [Gemini 2.5 Flash-Lite](#) · retirement 20 Oct 2026, regions
- [Gemini 3.1 Flash-Lite](#) · global, US and EU only
- [Gemini 3.5 Flash](#) · available in asia-south1
- [Vertex AI pricing](#)
- [Vertex AI locations and the global endpoint](#)
- [Text embeddings](#) · dimension limits
- [Vertex AI SDK migration guide](#)
- [Cloud Run pricing](#)

- [Cloud SQL pricing](#)
 - [Google Cloud pricing calculator](#)
-

Source document: `GCP_DEPLOYMENT_PLAN.md` in the Rusty repository. Figures are estimates; confirm them in the pricing calculator before you commit.